User Manual for PSM4ODES

Richard D. Neidinger
Davidson College
rineidinger@davidson.edu

March 14, 2025

PSM4ODES software generates series recurrence code and high-order Taylor solutions for Ordinary Differential Equations (ODEs) in MATLAB. PSM stands for Power Series Method (or arbitrary order Taylor series) to solve any ODE system of form $y' = f(t, y)$ and $y(t_0) = y_0$. It accompanies the article [N] *Automatic Series Recurrence Relations for Ordinary Differential Equations.*

1 Quick Start

1.1 Set Path and specify an ODE

Folder `PSM4ODES\Src\Functions` contains the software codes and should be added to the MATLAB path by `addpath` command or Set Path environment button. Then series recurrence code and numerical solution may be generated for an ODE system of form $y' = f(t, y)$.

Define a DE file, say `f.m`, as is usually done for any MATLAB ODE solver, where `f` takes and returns column vectors. For example, $y'' = \sin(y^2)$ uses $y_1 = y$ and $y_2 = y'$ in the system $y_1' = y_2$ and $y_2' = \sin(y_1^2)$. Avoid preallocating output with zeros or explicit doubles. This `f.m` file

```
function dydt = f(t,y)
dydt = [y(2); sin(y(1)^2)];
```

works well. Codes that start with `dydt = zeros(2,1)` will fail but may be made compatible by changing the preallocation to `dydt = y`, so that `dydt` replicates the class of `y` in the overloaded execution of `f` that generates the series recurrence code. The names of the variables in `f.m` can be anything though `t` an `y` will be used in the generated series code. We expect and implement only scalar functions (so `^` not `.^` in MATLAB). We now assume that an ODE of interest is given in a compatible file `f.m` or whatever name is chosen.

1.2 ODE software syntax

- `odepsmh.m` solves ode by series to order `deg` with step-size `h`.

  - Call: `[t,y] = odepsmh(@f,[t0,tf],h,y0,deg);`

- `odepsmJZ.m` solves ode by series, tolerance `tol` controls order and variable step size.

  - Call: `[t,y] = odepsmJZ(@f,[t0,tf],y0,tol);`

1.3 SERIES software (automatically called as needed by above)

- `makepsmcode.m` writes file `fseries.m` that generates coefficients for the solution of the ODE, where `numDEs` is the number of DEs in the system given by `f.m` or whatever name.

  - CALL: `makepsmcode(@f,numDEs)`

- `fseries.m` returns series coefficients about any initial point to order `deg`. After above call, this function will appear in the current folder containing `f.m`. If call above used `myDE.m` instead of `f.m`, then the function `myDEseries.m` would appear.

  - CALL: `coefs = fseries(t0,y0,deg)`

- `serieseval.m` evaluates series at t values `ts` using coefs about `t0`.

  - CALL: `values = serieseval(coefs,t0,ts)`

- `trace.m` is a class of objects used by `makepsmcode`. When `t` and `y` are trace objects, each operation (arithmetic or transcendental) in evaluation of `f(t,y)` is overloaded (as defined in `trace.m`) to write the corresponding code for the series recurrence relation including the evaluation giving the constant term.

2 Functions and Class

The reader may skip Sections 2.1 and 2.2 if primarily interested in usage of generated series coefficient code and ODE solutions. These sections supplement, and are best understood in conjunction with, [N].

2.1 makepsmcode

Function `makepsmcode` takes a file defining an ODE system and writes a corresponding MATLAB `.m` file that generates series coefficients for the solution of the ODE. The full input and output options are

```
[newfunc, valuelist, fvalue, location] ...
    = makepsmcode(fhandle, numDEs, t0, y0)
```

where the only required parameters are `fhandle`, a function handle to the `.m` file defining the ODE (for example, `@myDE` is the handle to `myDE.m`), and `numDEs` is the number of DEs in the ODE system. A useful output is `newfunc` which is a function handle (example `@myDEseries`) to the generated function file (example `myDEseries.m`) created or overwritten by the call. Even if a call has no actual output parameters, `makepsmcode` will print a notice that the file (example `myDEseries.m`) has been created or overwritten in the current folder, which is often all that is needed. If `fhandle` is an anonymous function, see the paragraph in section 2.4.1.

Other parameters reflect that one evaluation, as in `fvalue = myDE(t0,y0)`, is the mechanism behind `makepsmcode`. This evaluation is overloaded to keep track of every intermediate value, returned in `valuelist`, after each step (arithmetic operation or scientific calculator function). Input `y0` and outputs `fvalue` and `location` are vectors (of length numDEs), where `location` contains the indices where each component of `fvalue` is found in the `valuelist`. The real purpose of `makepsmcode` is not these numerical values, but that each overloaded step of evaluation also writes character strings for the evaluation (constant term of series) and the series recurrence for the operation [N]. Function `makepsmcode` then puts these together (using `location` indices internally) to write the series recurrence code for the ODE solution. This generated code works for any initial values. When `t0` and `y0` are not specified input to `makepsmcode`, random values between 0 and 1 are used for every component. These optional input and output parameters are allowed to aid in testing and understanding the process. They might also be needed in the case of an insurmountable domain problem with `t0` and `y0`, although we have never encountered this since MATLAB calculates complex values and carries `Inf` and `NaN` without crashing, and none of these values matter for the purpose of writing the code! Their output here is for testing and understanding.

2.2 trace class

The class trace.m is used by `makepsmcode` to accomplish all of the overloading described in the above paragraph. Every overloaded step of the evaluation uses trace object input and trace object output and calls the method `count_val`. Class Static function `count_val` has four persistent variables

`count`, `vallist`, `oplist`, and `reclist`,

that are updated with each overloaded evaluation call, incrementing operation `count` and appending to vector `vallist` (`valuelist` above), `oplist` (cell array of string(s) for performing the operation), and `reclist` (cell array of string(s) for performing the series recurrence).

Each trace class object has three properties:

`val`, `opcount`, and `tlinear`.

The numerical (or symbolic) value is stored in `val`, while `opcount` stores the current count indicating the step number. Boolean property `tlinear` is not necessary but, if true, allows for writing more efficient code when using an object whose series has at most two nonzero coefficients.

The trace class constructor method has syntax

```
obj = trace(value, role)
```

and initializes the three properties, varying with the role of each created instance. By default, when `trace` is called with no arguments or unless stated otherwise below, the properties `val` and `tlinear` are initialized as empty vectors `[]` and opcount as `NaN`. Since a positive `opcount` of a trace object will indicate that it comes from that numbered step of the evaluation, we use zero, negative, and `NaN` `opcount`s to indicate initial values and constants. Specifically, `makepsmcode` calls `t = trace(t0,0)` to initialize the independent variable indicated by `opcount` 0, with `val` `t0`, and `tlinear` `true`. It calls `y = trace(y0,1)` to initialize the dependent `y` as a vector of trace objects with `opcount -i`, `tlinear false`, and `val y0(i)` for the ith component. An overloaded operation (or scientific function) performs the usual computation resulting in `value` and calls `trace(value,2)` (or just `trace(value)`) to create an output object with `val` being `value`. After initial construction, the overloaded method calls `count_val` to assign the `opcount` and assigns `tlinear` by context. The default opcount `NaN` is used to flag a constant for the special case of a trivial DE like $y_i' = 1$. This case can be implemented in the DE file by `dydt = [1, y(2)]`, or `dydt = y` and `dydt(1) = 1`, but be careful not to use `dydt = ones(2,1)` which would overwrite the class of `dydt`. Otherwise, numerical constants do not need to be trace objects. Binary operations, such as `1+y(2)`, can be handled by the overloaded method, such as `plus`, in the trace class. Implementation of these trace constructor cases involved the following two facts. Creating an array of trace objects implicitly calls the default no argument case as starting property values. Assigning a constant, say 1, to one component of an array of trace objects implicitly calls `trace(1)`.

An overloaded method is defined in the trace class for every standard operation and scientific function when it is called with at least one trace object argument. More details about these algorithmic ideas are discussed in [N] and can be studied in the source code itself.

2.3 Direct use of generated series code

Command `makepsmcode(@myDE, numDEs)` writes or overwrites `myDEseries.m` file, which can be called by

```
coefs = myDEseries(t0,y0,deg)
```

where `t0` and `y0` are initial conditions for the ODE system whose solution has series coefficients computed up to degree `deg`. Both `y0` and `coefs` have `numDEs` rows for the component functions in the solution y. Because degree zero is the constant coefficient, `coefs` will have `deg`+1 columns for the coefficients.

For convenience in evaluating these component polynomials, we include a simple function based on Horner's rule. The syntax of the call to evaluate the truncated series polynomial(s) is

```
ys = serieseval(coefs,t0,ts)
```

where `coefs` about initial `t0` come from a call to the generated series code. Evaluation is at time(s) in `ts` that can be one value or a row vector of times. Output is a matrix of double values with one row for each component of the solution y and columns corresponding to times in `ts`. With appropriate span of times, `plot(ts,ys)` would plot the truncated series solution polynomials. Users may want to transpose `ys` into column vectors as in ODE solver output.

See `fex1_Script` and `fex2_Script` for examples.

2.3.1 Symbolic Toolbox for symbolic series and vpa coefficients

The generated series code will accept symbolic initial values that are `sym` (for letters or exact expressions) or `vpa` (variable precision arithmetic), if the MATLAB installation includes the Symbolic Toolbox. For an ODE with `numDEs = 1`, the generated code `myDEseries(t0,y0,deg)` produces a row vector of `deg+1` series coefficients.

If `y0 = 0.1`, then coefficients are doubles.

If `y0 = sym('a')`, then coefficients are symbolic expressions in `a`.

If `y0 = 1/sym(10)`, then coefficients are numerically exact expressions.

If `y0 = 1/vpa(10)`, then coefficients have 32 significant decimal digits.

Function `vpa(x,d)` will have d significant decimal digits, where `d = 32` is the default quadruple precision. Functions `sym` and `vpa` can convert simple fractions or decimals exactly, like `sym(.1)` is `1/10`, but others, like `100000/sym(1234567)`, require this more careful input. So it is best to convert one integer to `sym` or `vpa` first, then apply operations. See `fex1_SymbolicScript` for specific examples. WARNING: High order (say 20) series coefficients using `sym` can take a long time and be massive expressions, although using `vpa` is usually fast for just one set of coefficients.

All the initial values can be `sym` and/or `vpa`. The forced damped pendulum example `fdpendulum.m` has two components in y and uses independent t. The series code generated by `makepsmcode(@fdpendulum,2)` can be called by

```
symcoefs = fdpendulumseries(sym('t0'),[sym('a'),sym('b')],3);
disp(string(symcoefs.'))
```

where the second line is a nice way for an overview of the results.

2.4 ODE Solvers using generated series code

2.4.1 Common input and output

ODE input is given by a function handle, such as `@myDE` for a file `myDE.m` (as described for DE file `f.m` in the Quick Start section 1.1). Both solvers, `odepsmh` and `odepsmJZ`, check to see if the associated series code, such as `myDEseries.m`, is a file in the path or current folder. If not, the solver first calls `makepsmcode` (using length of `y0` for `numDEs`) to create the series code. Otherwise, it uses pre-existing series code for the solution. WARNING: The user must delete the

associated series code file whenever the original DE file is changed, otherwise the solution will correspond to the old version.

Anonymous functions are also allowed as a function handle for the ODE. The example `@(t,y) 2*t` , for $y' = 2t$, is the first example on the MATLAB help page for `ode45`. If an anonymous function is input to `odepsmh`, `odepsmJZ`, or `makepsmcode`, then `makepsmcode` will write or overwrite two files: `tempDE.m`, an explicit file equivalent to the anonymous function, and `tempDEseries.m`, the associated series code.

Common inputs to ODE solvers (here or supplied with MATLAB) include time span `[t0,tf]` and initial `y0`. Solutions can go in negative time direction by using `tf<t0`.

Variable precision arithmetic is triggered by a `vpa` initial value, as described in section 2.3.1, to both `odepsmh` and `odepsmJZ`. Be prepared to wait maybe seconds, minutes, or longer, depending on the number and complexity of steps being computed in variable precision arithmetic. MATLAB solvers do not allow `vpa` values. While `odepsmh` allows initial values using `sym`, it is not recommended for anything more than a few steps using lower degree.

Common output format of ODE solvers is `[t,y]`, where `t` is a column vector of times from `t0` to `tf` for each step of the solver. Each row in array `y` corresponds to the time in the corresponding row of `t`. The first column of `y` corresponds to y_1, and the second column corresponds to y_2, etc. Our PSM solvers return one y vector for each time step in the solver algorithm. This is common for most of MATLAB's solvers, but they do allow a `Refine` option, using `odeset`, that allows for multiple values on each step. In fact, ode45 uses a default setting of `Refine` 4 to return values for four equally spaced times on each step. For direct comparison, we specify `Refine` 1 for `ode45`. Our PSM solvers return only the ending polynomial value(s) after each step, although it would not be difficult to make new versions that evaluate the polynomial at a refinement of points within each step. Future versions could also output the computed coefficients on each interval, for a full piecewise representation of the curve and for analyzing coefficients.

2.4.2 odepsmh

The solver syntax

```
[t,y] = odepsmh(@myDE,[t0,tf],h,y0,deg);
```

computes a piecewise Taylor polynomial solution of the ODE initial value problem using a specified fixed step size `h` and fixed, but arbitrary, degree `deg`. New series coefficients are computed at each time step using the same unchanged series recurrence code generated by `makepsmcode`. After forming output vector `t` and checking for the series code, the solver is basically a loop with two steps: (1) use series code at the current `t(k)` and corresponding y values to compute coefficients of the polynomial(s) for each component and (2) evaluate the polynomial(s) at the next `t(k+1)` using Horner's rule (to build powers of `h`).

2.4.3 odepsmJZ

The solver syntax

```
[t,y,deg] = odepsmJZ(@myDE,[t0,tf],y0,tol);
```

computes a piecewise Taylor polynomial solution of the ODE initial value problem using tolerance `tol` to determine degree and adaptive step sizes. Optional output `deg` is the same degree for all the polynomials, chosen based on `tol`, specifically `deg = ceil(1-log(tol)/2)`. This and the adaptive algorithm for stepping are a version of those in [JZ] as explained near the end of [N]. The program initializes variables and constants, including `deg`, and checks for the series code. The time step loop has three parts: (1) use series code at the current `t(k)` and corresponding y values to compute coefficients of the polynomial(s) for each component, (2) use the coefficients to choose the size `h` for this step and next `t(k+1)`, and (3) evaluate the polynomial(s) at the next `t(k+1)` using Horner's rule (to build powers of `h`). The adaptive step (2) estimates the radius of convergence of the series, instead of the error on the step, using an adaptation of the root test on the computed coefficients. The chosen step is a fraction $(1/e^2)$ of the estimated radius of convergence with other safeguards to prevent too large a step. Details are discussed in [N].

3 Examples

Folder `PSM4ODES\Src\Examples` contains `.m` files for differential equations and scripts that show use of the PSM4ODES software features. Example DEs match those in [N] along with a bonus of the flame equation.

3.1 Example 0: $y'' = t * y$, Airy's Equation

- `airyDE.m` defines a system for Airy's equation

```
function dydt = airyDE(t,y)
dydt = [ y(2); t*y(1) ];
```

- `airyDE_Script.m` shows use of series coefficients and `odepsmJZ`.

The purpose of this example in [N] is to develop the algorithmic approach to the recurrence relation using symbolic manipulation and pseudocode by hand, while Example 1 introduces the key concept of a Cauchy Product. This script shows use of this software package on the Airy equation. The series code `airyDEseries.m` is written by makepsmcode. The pseudocode structure is evident in lines for "Evaluate" and "recurrence rules," noting the shift to index origin one in MATLAB so $Y_1[0]$ is stored in `Y(1,1)`. Also, the pseudocode recurrence $U_1[k] = T[0]Y_1[k] + T[1]Y_1[k-1]$ is a special case of a Cauchy product where the T vector only has two nonzero terms. Our automatically generated code leaves the full Cauchy Product `T(1,j:-1:1) * Y(1,1:j).'` which

is equivalent to `T(1,1)*Y(1,j)+T(1,2)*Y(1,j-1)`. Because MATLAB optimizes matrix multiplication, the Cauchy Product has similar efficiency to the two terms. The trace class does keep track that `T` is a linear variable, but only replaces the Cauchy Product when it devolves into just one term (as is the case when using a Derivative Cauchy Product [N]).

The initial condition `t0 = 0` and `y0 = [1;0]` leads to computation of the first of two linearly independent solutions of Airy's equation in Section 5.2 of [BD] (`[0;1]` leads to the other), as can be verified by the numerical coefficients displayed by this script. If installed, the Symbolic Toolbox uses initial `y0=[sym('a'),sym('b')]` to display the rational coefficients of both solutions. The piecewise degree 12 solution is computed by `odepsmJZ` and shown along with plots of some Maclaurin polynomials.

3.2 Example 1: $y' = \sin(y^2)$

- `fex1.m` defines the first order differential equation.

```
function dydt = fex1(t,y)
dydt = sin(y^2);
```

- `fex1_Script.m` shows use of series coefficients and `odepsmJZ`.

This generates Figure 2.1 in [N]. After giving reasonable initial conditions and constants (`t0 = 0`; `tend = 12`; `tspan = [t0,tend]`; `y0 = 0.1`; `tol = 1e-9`; `deg = 25`), the series code is generated. Values of one Maclaurin series are computed for different degrees and an `odepsmJZ` solution is computed. Illustrative lines:

```
makepsmcode(@fex1,1);  % writes fex1series.m code
coefs = fex1series(t0,y0,deg);
ts = linspace(t0,tend,100);
y10 = serieseval(coefs(1:11),t0,ts);  % values of degree 10 poly
[tpsm,ypsm,degfortol] = odepsmJZ(@fex1,tspan,y0,tol);
```

Results are plotted to make the figure. The number of steps are reported, using the same tolerance `tol`, for `ode45` and `odepsmJZ`.

- `fex1_SymbolicScript.m` computes symbolic and vpa coefficients and vpa solution.

This program requires that the Symbolic Toolbox is part of the MATLAB installation. A run echoes the commands that generate and display a few symbolic coefficients. First using `y0 = sym('a')` for truly symbolic coefficients, then `y0 = 1/sym(10)` for an exact numerical expression, then `y0 = 1/vpa(10)` for quadruple precision. The symbolic result in `a` is reported in Section 4 of [N].

With echo off, `y0 = 1/vpa(10)` is used in odepsmh and odepsmJZ and the final values are reported just to verify that these work with vpa. The

odepsmh call is equivalent to `odepsmh(@fex1,[0,12],1/8,1/vpa(10),20)`, so that it uses $h = 1/8$ and degree 20. The odepsmJZ call is equivalent to `odepsmJZ(@fex1,[0,12],1/vpa(10),1e-24)`, so that it carries 24 significant digits through the calculations. Each take around 20 seconds on my computer.

3.3 Example 2: $y'' = \sin(y^2)$

- `fex2.m` defines the system for the second order differential equation, see Quick Start Section 1.1.
- `fex2_Script.m` shows use of y and y' series coefficients and odepsmJZ.

This script generates a figure in the phase plane showing `ode45` and `odepsmJZ` solutions along with values of one Maclaurin series for different degrees. The DE file `fex2.m` is mentioned in [N] to discuss how an ODE system differs from a first order DE, specifically in `makepsmcode` output `fex2series` and its usage. However, the example computation and phase plane plot of this script is not in [N]. The script mimics `fex1_Script` but using initial `y0=[1;0]`. The same lines

```
coefs = fex2series(t0,y0,deg);
ts = linspace(t0,tend,100);
y10 = serieseval(coefs(1:11),t0,ts);  % values of degree 10 poly
```

now produce two rows in `coefs` and `y10` values corresponding to $y_1 = y$ and $y_2 = y'$. Matrix `y10` is transposed in the script to make it the same form as output from the ode solvers, specifically each column corresponds to a component of y and each row corresponds to a time.

3.4 Example 3: $y'' = -\sin(y) - y'/10 + \cos(t)$, Forced Damped Pendulum

- `fdpendulum.m` defines the system for this differential equation.

```
function dydt = fdpendulum(t,y)
dydt = [ y(2); -sin(y(1)) - 0.1*y(2) + cos(t)];
```

- `fdpendulum_Script.m` to graphically compare `odepsmh` with `ode45`.

This generates Figure 6.1 in [N]. This chaotic dynamical system is sensitive to the initial condition `y0 = [0;2]`. With default options, `ode45` does not find the correct "well" where the second half of the time span `[0,200]` should oscillate around the 6π pendulum angle. With fixed large step size 0.6 and using degree 20, `odepsmh` is able to find the correct orbit and end up within `2.8e-06` of the correct ending point, as found using the smallest allowable `RelTol` in `ode45`. That `odespmh` solution uses only 334 steps, while the accurate `ode45` solution uses 28,778 steps. Using the same fixed large step size 0.6 in the classic Runge-Kutta 4th order method follows a little closer (than default ode45) up to about time 60 but ends up in the wrong "well" as well.

- `fdpendulum_Timing.m` compares runtime for different solvers.

This script generates a table, following Figure 6.1 in [N], that compares errors and relative timings of the solutions from `fdpendulum_Script` plus three more solutions using `odepsmh` and `odepsmJZ`. Since runtimes can vary over different calls, this script repeats each solver for `numreps = 10` calls and finds an average runtime. Since timings can vary by machine, the timings are reported as a percentage of the runtime for `ode45` using tolerance `tolmin = 2.22045e-14`, the smallest allowable `RelTol`. Reported errors are the absolute error in the ending value of this sensitive system, where a hard-coded best ending value, that has about 20 significant digits, was found using `vpa` and `odepsmh` as shown in code comments. The presence of `0.1` in `fdpendulum.m` does not corrupt the `vpa` calculation since `vpa(.1)` does have the full quadruple precision, but we are playing with fire here since `vpa(.1234567)` does not, as can be verified by subtracting the fraction of vpa integers.

Using ordinary double precision, `odepsmh`, with step size 0.25 and degree 20, had absolute error `1.5e-13` which is relative error `8.6e-15`. The "accurate `ode45`" run used `tolmin` as both the `RelTol` and `AbsTol` and achieved absolute error `3.1e-12` (relative `1.8e-13`). When `odepsmJZ` is called with `tolmin`, it uses degree 17 and mean step size 0.2506 to achieve absolute error `9.9e-14` (relative `5.7e-15`). These accurate `odepsmh` and `odepsmJZ` runs are slightly faster than that accurate `ode45` and use vastly fewer steps. Finally, `odepsmJZ` with tolerance `1e-3` succeeds in finding a solution within visual accuracy using only degree 5 and 690 steps, while the default `ode45` fails to find this visual accuracy. The `fdpendulum_Timing` script can run in less than 30 seconds. Using `vpa` to find the best ending value can take 8 minutes or more.

3.5 Example 4: $y' = y^2(1-y)$, the Flame Equation

- `fflame.m` defines this stiff first order differential equation.

```
function dydt = fflame(t,y)
dydt = y^2*(1 - y);
```

- `fflame_Script.m` compares solvers using low tolerance.

This computes solutions of this difficult stiff equation with settings that show a remarkably good performance by `odepsmJZ`. It also has four lines that show computing `sym` and `vpa` coefficients if Symbolic Toolbox is installed. The flame equation and its challenges are discussed in [M]. It models the lighting of a match, with solution curve abruptly changing from near zero `y0` to a plateau value approaching 1 at time `1/y0`. For `y0 = 1e-4`, zooming on the sharp upper corner shows the stiffness effect in the default ode45 solution. With tolerance `1e-14`, `odepsmJZ` gives an accurate solution using degree 18 in only 70 steps! Using the smallest allowed RelTol of `2.22045e-14` (and the same AbsTol), ode45 was fast but used 4,535 steps. At this low tolerance, even stiff solver `ode15s` used 2,732 steps and `ode23s` used 81,772 steps! For larger

tolerances `odepsmJZ` does not perform as well; just raising 1e-14 to 2e-14, uses degree 17 and causes steps to jump from 70 to 1,353. For cruder tolerances, such as `1e-6`, the stiff solvers become much more efficient but the time of the abrupt vertical increase is off, as it is for `odepsmJZ` but not by as much. The magic of `1e-14` in `odepsmJZ` needs further study, perhaps degree 18 makes a difference. A promising future development could be adapting the degree along with the step size on each step.

3.6 Auxiliary Function

- `rk4.m` classic Runge-Kutta order 4 method for comparison.

This is an implementation of the simple classic Runge-Kutta order 4 method using fixed step size, provided as a convenience for comparing performance with `odepsmh` using any chosen degree.

References

[BD] W. E. Boyce and R. C. DiPrima, *Elementary Differential Equations and Boundary Value Problems*, 10th ed., Wiley, Hoboken, NJ, 2012.

[JZ] A. Jorba and M. Zou, *A Software Package for the Numerical Integration of ODEs by Means of High-Order Taylor Methods*, Experimental Math., 14 (2005), pp. 99-117, doi: 10.1080/10586458.2005.10128904.

[M] C. Moler, *Stiff Differential Equations*, MathWorks Technical Article, 2003, https://www.mathworks.com/company/technical-articles/stiff-differential-equations.html.

[N] R.D. Neidinger, *Automatic Series Recurrence Relations for Ordinary Differential Equations*, preprint 2025.